📋 Relatório de Auditoria Crítica: Agentes de IA e Métricas

Projeto: Poker Texas Hold'em - Simulação e Detecção de Blefes (TCC)

Auditor Responsável: Rigor (*Performance & Data Auditor*)

Data da Auditoria: 30 de Outubro de 2026

Status do Ecossistema: ⚠️ Anomalias Críticas Detectadas

---

1. Resumo Executivo

Esta auditoria foi realizada com postura estritamente cética e orientada à integridade científica do Trabalho de Conclusão de Curso (TCC). Foram inspecionadas as implementações dos agentes em `src/agents/`, a infraestrutura de telemetria e geração de dados em `src/telemetry/` e `src/database/`, os pipelines de treino em `src/training/` e os scripts de benchmark e visualização em `scripts/`.

Principais Constatações

1. **Explorabilidade Máxima de Agentes:** O `BlufferAgent` possui $P(\text{Blefe} \mid \text{Raise}) = 1.0$ (nunca aumenta por valor), enquanto o `MathematicalAgent` atua como *Calling Station* no River pagando apostas com qualquer carta alta (ex: 7-2o).
1. **Descarte Crítico de Mãos Premium:** O `HybridAgent` dá fold pré-flop com mãos de elite como $AK\text{s}$, QQ e JJ em mesas 6-max devido a distorções no cálculo de equidade contra múltiplos oponentes.
1. **Métricas Gráficas e Benchmarks Corrompidos:** O gráfico de evolução de bankroll (`plot_bankroll_evolution`) acumula exclusivamente fichas de blefe (`net_bluff_chips`), zerando 100% dos lucros e perdas de mãos legítimas de valor. No script de benchmark comparativo, textos exibem falsamente vantagens invertidas.
1. **Vieses Severos nos Modelos Treinados:** Modelos salvos exibem anomalias estatísticas flagrantes, como um agente de blefe classificado como "Tier S" que venceu 94% das mãos sem jamais ter tentado um único blefe.

---

2. Auditoria Detalhada dos Agentes (`src/agents/`)

🔴 2.1. `BlufferAgent` (`src/agents/bluffer_agent.py`)

Gravidade: Alta | Categoria: Teoria dos Jogos & Lógica de Decisão

1. **100% dos Raises são Blefes (Zero Value Raises):**

* No método `_decide_value_play` (`bluffer_agent.py`, linhas 817-825), quando o agente possui uma mão dominante (AA, KK, Trinca, Flush, Royal Flush) e enfrenta uma aposta (`to_call > 0`), ele **sempre dá CALL**; nunca re-aumenta por valor.

* Os únicos caminhos que geram `ActionType.RAISE` no código são `_evaluate_semi_bluff`, `BLUFF_RAISE` e `PREFLOP_STEAL`.

* **Vício Estrutural:** Para qualquer oponente ou classificador de IA, a probabilidade condicional de blefe dado um raise é de 100%:

$$P(\text{Blefe} \mid \text{Raise}) = 1.0$$

O agente é completamente previsível: adversários podem pagar ou dar reraise em todos os seus aumentos com certeza matemática de estarem enfrentando um blefe.

1. **Comportamento "Calling Station" Passivo:**

* **Pré-Flop:** se `to_call <= big_blind`, ele entra de call em 100% das mãos. Diante de aumentos pré-flop:

```python
if (is_speculative or roll < 0.60): return CALL
```

O agente paga 60% das vezes às cegas segurando combinações desprovidas de equidade (ex: 7-2o, 8-3o).

* **Pós-Flop:** paga apostas de até 120% do pote em 60% das vezes às cegas (`bluffer_agent.py`, linha 575), dilapidando fichas sem fundamentação matemática.

🔴 2.2. `MathematicalAgent` (`src/agents/mathematical_agent.py`)

Gravidade: Alta | Categoria: Lógica Probabilística & Teoria de Decisão

1. **Calling Station no River com Carta Alta (Lixo Puro):**

* No método `_decide_facing_cost` (`mathematical_agent.py`, linha 494):

```python
elif is_mod_bet and (outs >= 4 or street == Street.RIVER):
is_profitable = True
```

* Diante de qualquer aposta moderada ($\le 45\%$ do pote ou $\le 3\text{ BB}$), no **River**, ele crava `is_profitable = True` independentemente do valor da mão.

* **Consequência:** O agente supostamente guiado por valor esperado estrito paga apostas no River sem par algum (ex: 7-2 offsuit ou Ás alto sem kicker).

1. **Dupla Contagem e Inflação Artificial de Equidade:**

* No método `_calculate_draw_outs`, outs de flush (9), sequência (8) e overcards (6) são somados linearmente (podendo atingir até 23 outs teóricos).

* Em seguida, calcula:

$$\text{adjusted_equity} = \max(\text{equity}_{\text{MonteCarlo}}, \text{draw_prob} \times \text{draw_weight})$$

* O algoritmo de Monte Carlo `estimate_hand_equity` **já simula o bordo completo até o River** e já contempla as probabilidades de completar draws. Sobrepor a regra empírica de 4 e 2 causa dupla contagem e ignora riscos de redraws adversários.

1. **Simulação Contra Ranges Uniformes Aleatórios:**

* As simulações assumem cartas 100% aleatórias para os oponentes, ignorando que oponentes que apostam ou aumentam possuem ranges filtrados e substancialmente mais fortes.

🔴 2.3. `HybridAgent` (`src/agents/hybrid_agent.py`)

Gravidade: Alta | Categoria: Estratégia de Ranges & Telemetria

1. **Fold Sistemático de Mãos Monstro Pré-Flop em Mesas 6-Max:**

* O agente não utiliza tabela posicional ou combinatória pré-flop; executa simulação Monte Carlo pura contra N oponentes (`hybrid_agent.py` , linha 306).

* Em mesas de 6 jogadores ($N=5$), a equidade individual média de mãos fortes fica entre 27% e 38% ($AK\text{s} \approx 33\%$, $QQ \approx 38\%$, $JJ \approx 32\%$).

* No pré-flop, cobrir o Big Blind (20 fichas em pote de 30) exige:

$$\text{Pot Odds} = \frac{20}{30 + 20} = 40\%$$

* Como a equidade simulada (33%) é inferior ao limiar exigido ($40\% - 2\%$), o agente conclui que pagar é -EV e executa **FOLD pré-flop segurando AKs, QQ e JJ**.

1. **Incompatibilidade de Esquema de Telemetria:**

* O `HybridAgent` registra suas jogadas de engano como `is_deceptive` , enquanto o módulo de telemetria e o `AdaptiveTrainer` buscam pela chave `is_bluff` . Isso faz com que seus blefes não sejam computados nas estatísticas globais.

🟡 2.4. `GhMathAgent` (`src/agents/gh_math_agent.py`)

Gravidade: Média | Categoria: Modelagem Probabilística

1. **Cálculo de Odds Implícitas no River:**

Em `_calculate_implied_odds` , o agente calcula retorno futuro potencial mesmo durante o River. No River, não há streets futuras nem cartas a serem reveladas; implied odds* são teoricamente nulas por definição.

1. **Distorção Aditiva de Posição:**

* O bônus posicional é adicionado diretamente à equidade bruta ($\text{Equity} + \text{Bonus}$), inflando a fração do Critério de Kelly e induzindo a apostas desproporcionais ao valor real da mão.

🟡 2.5. `GhBlufferAgent` (`src/agents/gh_bluffer_agent.py`)

Gravidade: Média | Categoria: Calibração de Frequências

1. **Hiper-agressividade em Bordos Monótonos:**

* Em bordos com 3 ou 4 cartas do mesmo naipe, a presença de um Ás de bloqueador é tratada como justificativa absoluta para blefes no River, aplicando a razão de indiferença GTO como frequência direta em vez de mista/estocástica.

🟡 2.6. `HeuristicAgent` (`src/agents/heuristic.py`)

Gravidade: Média | Categoria: Heurística Básica

1. **Avaliação Desconectada da Textura da Mesa:**

* Avalia a mão estritamente por categoria nominal: um Par de 2 em bordo $A\text{-}K\text{-}Q\text{-}J$ recebe a mesma pontuação base que um Par de Áses em bordo seco $2\text{-}7\text{-}J$.

1. **Blefes em Potes Multiway:**

* Executa blefes quando `hand_strength < 0.30` sem considerar o número de adversários ativos, tentando blefar mesas com 4 ou 5 oponentes onde a Fold Equity é praticamente nula.

---

3. Auditoria dos Pipelines, Treinamento e Scripts

🔴 3.1. Corrupção da Curva de Bankroll (`plot_championship_results.py` e `telemetry_generator.py`)

- **Local:** `src/database/telemetry_generator.py` (linhas 554-557) e `scripts/plot_championship_results.py` (linha 123).
- **Mecanismo da Falha:** A variável `net_bluff_chips` só é computada dentro da cláusula `if is_bluff:` . Para mãos vencidas ou perdidas por valor, check ou fold honesto, o valor é fixado em `0` . Em seguida, a função `plot_bankroll_evolution` usa exclusivamente essa métrica para traçar o gráfico de saldo acumulado.
- **Impacto: 100% dos ganhos e perdas legítimos são ignorados.** O gráfico oficial de "Evolução do Saldo de Fichas" do campeonato representa uma métrica distorcida e cientificamente inválida.

🔴 3.2. Inversão Textual de Resultados (`scripts/benchmark_gh_agent.py` e Relatório)

- **Local:** `scripts/benchmark_gh_agent.py` (linha 307) e `BENCHMARK_GH_AGENT_REPORT.md` .
- **Mecanismo da Falha:** O código calcula `gh.bb_per_100 - alb.bb_per_100` e anexa textualmente `"a favor de Gustavo"` . Como o resultado foi -786.26, imprimiu:

> `-786.26 BB/100 a favor de Gustavo`

Além disso, calculou `-56.9% de melhoria` em folds no pré-flop, quando na verdade o agente dobrou a frequência de folds (de 39.4% para 61.8%).

- **Impacto:** O relatório textual afirma vantagem para um agente quando o agente adversário obteve performance 70 vezes superior em lucratividade real (+797.40 vs +11.14 BB/100).

🔴 3.3. Exclusão Precoce de Agentes no Treinamento (`src/training/adaptive_trainer.py`)

- **Local:** `src/training/adaptive_trainer.py` (linha 218).
- **Mecanismo da Falha:** A recarga de fichas só é acionada quando `len(active) < 2` . Se o agente candidato sob treinamento perde todas as fichas (`stack = 0`), mas restam 2 ou mais oponentes ativos, o candidato é excluído da mesa e as mãos restantes da simulação continuam sem a sua participação.
- **Impacto:** As métricas de BB/100 e taxa de vitórias são calculadas dividindo-se pelo total nominal de mãos da sessão, subestimando drasticamente a variância e gerando dados incompletos.

🔴 3.4. Inconsistência nos Modelos Treinados Salvos

- **`models/Blef.json` :** Classificado com Tier S (Elite), fitness 100.0 e Win Rate de 94.0% em 50 mãos (47 vitórias em mesa de 6 jogadores, o que é estatisticamente inverossímil). Ao inspecionar os atributos:

$$\text{bluffs_attempted} = 0 \quad | \quad \text{bluff_success_rate} = 0.0 \quad | \quad \text{fold_equity_realized} = 0.0$$

Trata-se de um modelo qualificado como "Mestre do Blefe" que **nunca executou um blefe sequer**.

- **`models/Math.json` :** Registra taxa de ganho de **+1.458,2 BB/100** em 50 mãos. Em mesas reais, jogadores de nível mundial operam entre 5 e 10 BB/100. Uma taxa de 1.458 aponta saturação por adversários doando fichas de forma passiva.

---

4. Matriz Diagnóstica Consolidada

Componente Auditado	Natureza do Defeito	Gravidade	Efeito Prático no Sistema
`BlufferAgent`	Ausência de Value Raises	🔴 Alta	Raises são 100% blefes; agente vira alvo fácil de contra-exploração.
`BlufferAgent`	Calling Station Cego	🔴 Alta	Paga 60% das apostas sem equidade pré e pós-flop.
`MathematicalAgent`	Calling Station no River	🔴 Alta	Paga apostas com carta alta (7-2o) diante de qualquer aposta moderada.
`MathematicalAgent`	Dupla contagem de draws	🔴 Alta	Equidade artificialmente inflada sobre o Monte Carlo.
`HybridAgent`	Fold de cartas premium	🔴 Alta	Desiste pré-flop de AKs, QQ e JJ em mesas cheias.
`HybridAgent`	Telemetria divergente	🟡 Média	Não propaga blefes para o `AdaptiveTrainer` (`is_deceptive`).
`GhMathAgent`	Implied odds no River	🟡 Média	Projeta expectativas futuras em rodada terminal.
`telemetry_generator`	Filtragem de bankroll	🔴 Alta	Gráfico oficial ignora lucros e perdas de mãos sem blefe.
`benchmark_gh_agent`	Inversão de strings	🔴 Alta	Textos de relatório afirmam vantagem do agente perdedor.
`AdaptiveTrainer`	Falha na recarga (re-buy)	🔴 Alta	Agente eliminado não recarrega se sobrarem 2 bots.
Modelos em `models/`	Vício de amostragem	🔴 Alta	"Mestre do blefe" com 0 blefes; BB/100 hiper-inflado (+1458).

5. Recomendações e Plano de Ação Prioritário

Prioridade 1: Correção nos Agentes (Sprint de Correção de Decisão)

1. **Balancear `BlufferAgent._decide_value_play`:**

Adicionar lógica para que mãos fortes (Dois Pares, Trincas, Flushes e Full Houses) aumentem e re-aumentem por valor proporcional ao pote, restabelecendo a credibilidade das apostas do agente.

1. **Remover o Call Automático no River em `MathematicalAgent`:**

Excluir a condição `or street == Street.RIVER` na linha 494 de `mathematical_agent.py` e exigir no mínimo um Par Médio para justificar *bluff catching*.

1. **Calibração Pré-Flop no `HybridAgent`:**

Implementar tabela de força combinatória pré-flop (semelhante à utilizada pelo `GhMathAgent`), impedindo que o cálculo ingênuo de pot odds force folds com mãos premium.

Prioridade 2: Saneamento das Métricas e Visualizações

1. **Correção do Gráfico de Bankroll (`plot_championship_results.py`):**

Substituir o uso de `net_bluff_chips` pelo saldo líquido total da mão (`net_chips`) para refletir a verdadeira trajetória financeira dos agentes.

1. **Correção do Script de Benchmark (`benchmark_gh_agent.py`):**

Adequar a lógica condicional de impressão para que o diferencial de BB/100 e a redução de folds reflitam corretamente o autor/agente que obteve a real vantagem.

1. **Recarga Incondicional no `AdaptiveTrainer` :**

Garantir que, sempre que o stack do candidato sob teste atingir zero, o recarregamento ocorra independentemente do número de adversários restantes na mesa.

Relatório emitido pelo agente Rigor em conformidade com as diretrizes do Antigravity Orquestrador.